A DOCUMENTATION OF GENAI USAGE – PROMPTED CHATGPT (GPT-5.3)

death cab nlp project. explain how I can efficiently get data. should i scrape genius or is there an API? my goal is a csv with the lyrics all in one column, with other columns for album name, band name (death cab, postal service, ben gibbard), and song name?
Thought for 18s

Yes — the efficient route is **API for metadata, scrape for lyrics**.

The main catch is that Genius's API is good for finding artists, songs, albums, and song URLs, but it does **not cleanly hand you a "full lyrics" field as a normal API endpoint**. Libraries like `lyricsgenius` work by combining Genius API calls with pulls from Genius's public pages.

So for your project, I would **not** build a raw scraper first. I'd do this:

Best workflow

1. **Use Genius / `lyricsgenius` to get song metadata**
    - artist name
    - song title
    - album
    - Genius URL
2. **Use the same toolchain to fetch the lyrics text**
    - either through `lyricsgenius`
    - or by scraping the specific Genius song page if needed
3. **Save one row per song** into a CSV with columns like:
    - `band_name`
    - `album_name`
    - `song_name`
    - `lyrics`

That is the cleanest setup for NLP.

What I recommend

Use **Python + `lyricsgenius`** first.

Why:

- faster than hand-writing a scraper
- already built for Genius-style workflows
- can search artists and songs, and save artist/song data including lyrics content through its wrapper flow

Practical recommendation for your exact project

Since your artists are basically:

- Death Cab for Cutie
- The Postal Service

- Ben Gibbard

you do **not** need some giant industrial pipeline.

Just:

- create a small artist list
- pull songs for each artist
- clean out duplicates / live tracks / remasters / demos if needed
- write the final dataframe to CSV

Important reality check

Scraping lyrics sites can get messy fast because of:

- duplicate versions
- live/acoustic/remastered entries
- featured appearances
- annotations / extra text
- inconsistent album tagging

So the real work is **not getting any lyrics at all**.
The real work is **getting a clean corpus**. That is where your project quality will show.

Good CSV structure

I'd use:

- `artist_group` → one of `Death Cab for Cutie`, `The Postal Service`, `Ben Gibbard`
- `album_name`
- `song_name`
- `lyrics`
- `genius_url`
- `release_type` (optional: album, EP, soundtrack, single)
- `source_artist` (optional, if Genius returns weird collabs)

Example row shape:

artist_group	album_name	song_name	lyrics
Death Cab for Cutie	Transatlanticism	Tiny Vessels	full lyrics text...

Efficient strategy by project stage

Option A: fastest for class/project use

Use `lyricsgenius.search_artist(...)` for each artist, then loop through songs and build a dataframe.

This is the best first pass.

Option B: more controlled / cleaner

Build your own song list first, then query song-by-song.

This is slower up front, but better if you care about:

- only studio albums
- no duplicates
- no deluxe versions
- no features
- no random Genius clutter

For an NLP portfolio project, this is honestly often the better move.

My blunt recommendation

Do this:

- **Use `lyricsgenius`**
- pull artist/song metadata + lyrics
- manually clean the corpus afterward

Do **not** start by writing a custom BeautifulSoup scraper unless:

- `lyricsgenius` fails badly for your artists, or
- you specifically want to showcase web scraping

Because your project is **NLP on Ben Gibbard's songwriting**, not "I can suffer through brittle HTML parsing."

Example starter code

```
import pandas as pd
import lyricsgenius

GENIUS_API_TOKEN = "YOUR_TOKEN_HERE"

genius = lyricsgenius.Genius(GENIUS_API_TOKEN)
genius.skip_non_songs = True
genius.excluded_terms = ["(Live)", "(Remix)", "Remix", "Live", "Demo"]
genius.remove_section_headers = True
genius.timeout = 15
genius.retries = 3

artists = [
    "Death Cab for Cutie",
    "The Postal Service",
    "Ben Gibbard"
]

rows = []

for artist_name in artists:
    artist = genius.search_artist(
        artist_name,
        max_songs=None,   # or set a limit while testing
        sort="title"
```

```
    )

    if artist is None:
        print(f"Could not find artist: {artist_name}")
        continue

    for song in artist.songs:
        rows.append({
            "band_name": artist_name,
            "album_name": getattr(song, "album", None) if hasattr(song, "album") else None,
            "song_name": song.title,
            "lyrics": song.lyrics,
            "genius_url": song.url
        })

df = pd.DataFrame(rows)

# basic cleanup
df["lyrics"] = df["lyrics"].str.replace(r"^\d+ Contributors.*?Lyrics", "", regex=True)
df["lyrics"] = df["lyrics"].str.replace(r"You might also like", "", regex=False)
df["lyrics"] = df["lyrics"].str.replace(r"Embed$", "", regex=True)

df.to_csv("ben_gibbard_lyrics.csv", index=False)
print(df.head())
print(df.shape)
```

Notes:

Used the lyricsgenius idea

Took and modified starter code -> pulling.py